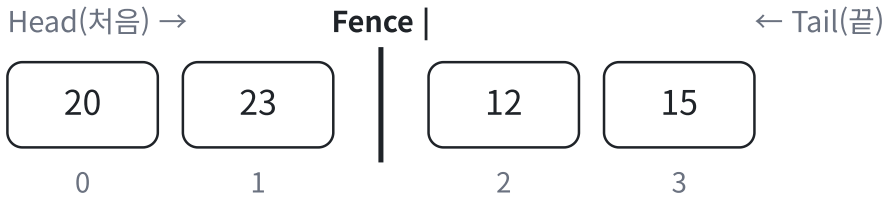
CSED233 · 데이터구조 · 포스텍

# 정리본 4. Lists, Stacks, Queues

선형 자료구조 3종: List ADT와 구현 방식, Stack(LIFO), Queue(FIFO), Deque.

## 1. List(리스트) — 순서가 있는 컬렉션

**List(리스트)**: 순서가 있는 유한 개의 원소 모음. L = ⟨ $a_1$, $a_2$, ⋯, $a_n$ ⟩. 중복 허용, 공 리스트(n=0) 가능.




L = ⟨ 20, 23, | 12, 15 ⟩ — Fence는 "처리 완료 구간"과 "미처리 구간"의 경계. Insert(10)은 Fence 위치에 삽입: ⟨ 20, 23, 10 | 12, 15 ⟩

**Sorted vs Ordered**

- **Sorted List(정렬된 리스트)**: 원소가 값 기준 오름차순. ⟨ 85, 88, 90, 92, 95 ⟩. 저절로 유지되지 않고 정렬이 필요.
- **Ordered List(순서 리스트)**: 삽입 순서만 유지, 값은 무관. ⟨ 90, 85, 95, 88, 92 ⟩.
- "순서가 있다(ordered)"는 말이 "정렬됐다(sorted)"는 뜻이 아니다.

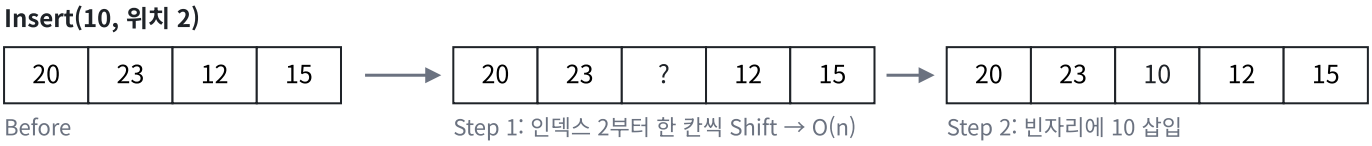
### 1.1 List의 기본 연산 8가지

| 연산 | 설명 | 시간복잡도 |
|---|---|---|
| Insert(x, p, L) | 인덱스 p에 x 삽입 | O(n) 또는 O(1) — 구현에 따라 |
| Delete(p, L) | 인덱스 p의 원소 삭제 | O(n) 또는 O(1) — 구현에 따라 |

| | | |
|---|---|---|
| Search(x, L) | 원소 x 찾기 | O(n) |
| Locate(x, L) | x의 인덱스 반환 | O(n) |
| Retrieve(p, L) | 인덱스 p의 값 반환 | 배열 O(1) · 연결 리스트 O(n) |
| Next(p, L) | p의 다음 인덱스 | O(1) |
| Previous(p, L) | p의 이전 인덱스 | O(1) |
| MakeNull(L) | L을 공 리스트로 | 구현에 따라 O(1)~O(n) (노드 해제 여부) |

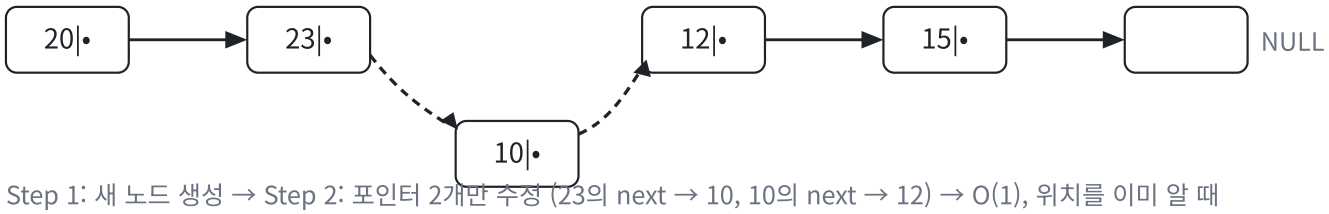
## 2. 구현: 배열 기반 vs 포인터 기반

### 2.1 배열 기반 — 연속 메모리




배열 삽입: Shift 비용 때문에 O(n). 맨 뒤 삽입만 O(1) (끝에 여유 공간이 있을 때; 재할당 시 O(n), 분할상환 O(1)).

### 2.2 포인터 기반 — 연결 리스트




연결 리스트 삽입: 데이터 이동 없이 포인터 수정만

**Dummy Header(더미 헤더)**

연결 리스트 맨 앞에 데이터가 없는 빈 노드를 두는 기법. 더미가 없으면 "첫 번째 노드에 삽입" 같은 경우에 `if (p == null)` 특수 분기가 필요하다.

구조: `L → [dummy] → a → b → c` — 모든 삽입/삭제가 같은 코드 경로를 타게 된다.

### 2.3 Cursor(커서) 기반 — 배열로 포인터 흉내

배열 인덱스를 포인터처럼 쓰는 기법. 인덱스 0은 NULL 역할로 예약한다.

| Index | Data | Next | 비고 |
|---|---|---|---|
| 0 | — | — | NULL 역할 (예약) |
| 1 | 'A' | 4 | L1의 첫 노드 |
| 2 | 'X' | 6 | L2의 첫 노드 |
| 3 | — | 5 | Free(빈 공간) 리스트 시작 |
| 4 | 'B' | 7 | L1의 둘째 노드 |
| 5 | — | 0 | Free 리스트 끝 |
| 6 | 'Y' | 0 | L2의 끝 |
| 7 | 'C' | 0 | L1의 끝 |

읽는 법: L1 = 1(A) → 4(B) → 7(C, next 0 = 끝). L2 = 2(X) → 6(Y). Free = 3 → 5.

### 2.4 성능 비교

| 연산 | 배열(Array) | 포인터(링크드) | 선택 기준 |
|---|---|---|---|
| Insert / Delete | O(n) | O(1) (위치 알 때) | 삽입·삭제 많으면 링크드 |
| Search / Locate | O(n) | O(n) | 동등 |
| Random Access | O(1) | O(n) | 접근 많으면 배열 |
| 메모리 | 연속 | 분산 | 캐시 효율은 배열 |
| 구현 | 간단 | 복잡 | 학습 곡선은 배열 |

## 3. Stack(스택) — LIFO

**Stack(스택)**: Last In First Out — 마지막에 들어온 것이 먼저 나간다. 현실의 접시 쌓기.




push(a) → push(b) → push(c) → push(d) 순으로 넣으면 pop()은 d부터

- push(x), pop(), top(), isEmpty() — 모든 연산 O(1).
- 응용: 함수 호출 스택(Call Stack — main → funcA → funcB, 종료는 역순), 괄호 검증( `([{}])` 올바름 / `([}])` 오류 ), Undo.

---

#### 예제 · Multiple Stacks — 배열 하나에 스택 둘

1차원 배열의 양 끝에서 마주 보며 자라게 하면 공간을 공유할 수 있다.

```
[ Stack1 →  ...빈 공간...  ← Stack2 ]
 Top1 →                ← Top2
 [0]                   [n-1]
```

**Overflow 조건: Top1 + 1 == Top2** (두 탑이 맞닿는 순간). 한쪽이 적게 쓰면 다른 쪽이 남은 공간을 쓸 수 있어 메모리 효율적.

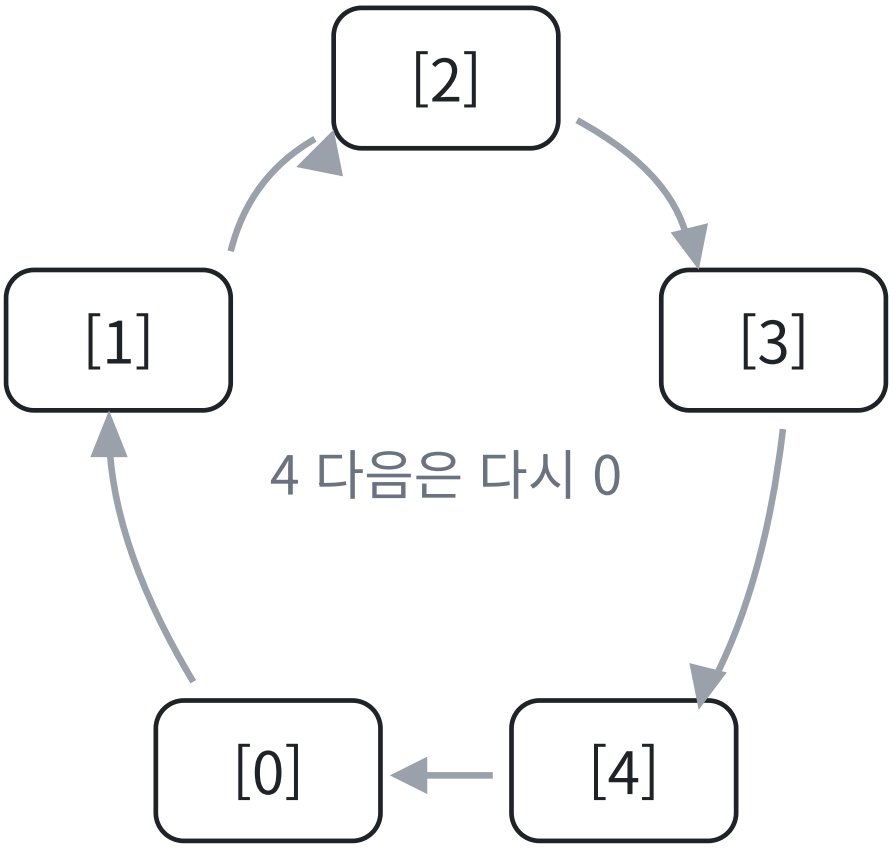
## 4. Queue(큐) — FIFO

**Queue(큐)**: First In First Out — 먼저 들어온 것이 먼저 나간다. 현실의 은행 번호표.

- `enqueue(x)` : Rear(뒤)에 삽입 · `dequeue()` : Front(앞)에서 제거·반환 · 모두 O(1).
- 응용: 프린트 대기열, 콜센터, BFS.

### 4.1 Circular Queue(원형 큐)

일반 배열 큐는 enqueue/dequeue를 반복하면 Front와 Rear가 계속 오른쪽으로 이동해 **앞에 빈칸이 있어도 Rear가 끝에 닿으면 삽입 불가**가 된다. 해결책은 배열 끝과 처음을 논리적으로 연결하는 것 — Modulo(나머지) 연산으로 인덱스를 돌린다.




원형 큐: 인덱스가 끝나면 처음으로

**인덱스 갱신 공식**

$$\text{Rear} \leftarrow (\text{Rear} + 1) \bmod size$$

$$\text{Front} \leftarrow (\text{Front} + 1) \bmod size$$

**Empty와 Full의 구분**

비었을 때도 **Front == Rear**, 꽉 찼을 때도 **Front == Rear** — 포인터만으로는 구분할 수 없다.
해결책(택 1):

1. **한 칸 비워두기** (가장 흔함): 아래 조건이면 Full로 판정.

$$(\text{Rear} + 1) \bmod size = \text{Front} \;\Rightarrow\; \text{Full}$$

2. **count 변수**로 원소 개수를 별도 추적.

#### 4.2 Linked Queue(연결 큐)

포인터로 노드를 체인처럼 연결: Front 포인터 → [Data|Next] → ⋯ → [Data|NULL] ← Rear 포인터.
배열 기반(원형 큐 포함)은 처음에 크기를 정해야 하므로, 데이터 양을 모를 때는 포인터 기반이 낫다.

## 5. Deque(덱) — Double-Ended Queue

양쪽 끝에서 삽입/삭제가 모두 가능한 큐. Stack + Queue의 기능을 합친 인터페이스로 보면 된다.

- 양쪽 끝의 삽입·삭제·원소 확인 모두 O(1) (임의 위치 접근은 구현에 따라 다름). 구현: 원형 배열 또는 이중 연결 리스트.
- 응용: 슬라이딩 윈도우 등 앞뒤 접근이 모두 필요한 경우.

## 6. 선택 가이드

**상황 → 자료구조**

- 최근 것부터 처리 → **Stack** (Undo, 함수 호출, 뒤로가기)
- 먼저 온 것부터 처리 → **Queue** (프린트 대기열, 콜센터)
- 양쪽에서 접근 → **Deque**
- 삽입/삭제가 잦다 → **Linked List**
- 인덱스로 빠르게 접근 → **Array**

## 7. Quiz

**Q1. Stack과 Queue의 가장 큰 차이는?**

힌트

"어느 쪽에서 나가는가"가 다르다: 스택은 넣은 쪽, 큐는 반대쪽.

정답

개념은 힌트 참고.

**정답: Stack은 LIFO(마지막에 들어온 것 먼저), Queue는 FIFO(먼저 들어온 것 먼저).**

### Q2. 배열 기반 List에서 맨 앞에 삽입하는 비용은?

힌트

섹션 2.1의 Shift 그림: 앞에 넣으면 모든 원소가 움직인다.

정답

개념은 힌트 참고.

① 맨 앞 삽입은 n개 전부를 오른쪽으로 한 칸씩 Shift해야 한다.

② 따라서 O(n). 맨 뒤 삽입만 Shift가 없어 O(1).

**정답: O(n) — 모든 원소를 Shift해야 한다. 맨 뒤 삽입만 O(1).**

### Q3. 연결 리스트에서 Insert는 O(1)인데 Locate는 O(n)인 이유는?

힌트

O(1) Insert에는 "위치를 이미 안다"는 전제가 붙는다.

정답

개념은 힌트 참고.

① Insert: 위치를 알면 포인터 2개만 수정 → O(1).

② Locate: 원소가 어디 있는지 모르니 처음부터 순회 → O(n).

**정답: Insert는 위치를 알 때 포인터만 바꾸면 되지만, Locate는 찾으려면 순회가 필요하기 때문.**

### Q4. Stack을 배열로 구현할 때의 장점과 단점은?

> 힌트

배열의 일반적 성질(연속 메모리, 고정 크기)을 스택에 대입해 보자.

> 정답

개념은 힌트 참고.

**정답: 장점 — 구현이 간단하고 접근이 O(1). 단점 — 크기가 고정되어 메모리 낭비 또는 Overflow 가능.**

### Q5. 왜 Deque을 따로 배우는가?

> 힌트

Stack도 Queue도 한쪽 끝만 빠르다. 양쪽이 다 필요하면?

> 정답

개념은 힌트 참고.

**정답: 양쪽 끝에서의 빠른 접근이 동시에 필요한 경우(슬라이딩 윈도우 등)가 있어서. Stack+Queue 기능을 하나로 제공한다.**

### Q6. Singly Linked List를 Doubly Linked List로 바꾸려면?

> 힌트

뒤로 가려면 뒤를 가리키는 포인터가 필요하다. 비용도 같이 생각해 보자.

> 정답

개념은 힌트 참고.

① Node 구조체에 `Node* prev` 필드를 추가한다.

② Insert/Delete 시 `prev` 포인터도 함께 갱신한다.

③ 대가: 노드당 포인터 1개분 공간 추가. 얻는 것: 양방향 탐색.

**정답: `prev` 포인터를 추가하고 갱신 로직을 보강한다. 공간은 늘지만 양방향 탐색을 얻는다.**

### Q7. Circular Queue에서 Front == Rear일 때 Empty/Full을 구분 못하는 근본 이유는?

> **힌트**

상태의 개수를 세어 보자. 크기 n 배열의 "채워진 정도"는 0개~n개로 n+1가지다.

> **정답**

개념은 힌트 참고.

① 크기 n 배열의 가능한 채움 상태는 n+1가지(0개~n개).

② 그런데 Front == Rear 조건만으로는 Empty와 Full이 같은 표현으로 보인다.

③ 그래서 한 칸을 희생하거나 count 변수로 상태 공간을 분리한다.

**정답: F == R 표현 하나에 Empty와 Full 두 상태가 겹치기 때문 → count 변수나 한 칸 희생으로 구분한다.**

### Q8. Cursor List에서 link = 0을 끝(sentinel)으로 쓰는 이유는?

> **힌트**

포인터 세계의 NULL을 인덱스 세계에서 무엇으로 대신할까?

> **정답**

개념은 힌트 참고.

**정답: 인덱스 0을 "유효 노드 없음"으로 예약해 NULL을 대체한다. 실제 노드는 인덱스 1부터 시작.**